CORE JAVA SERIES — DAY 3

Type Conversion & Operators

How Java converts between data types automatically and manually, then a full tour of arithmetic, relational, bitwise, shift, logical, and assignment operators.

Type Conversion: The Two Kinds

Type conversion means moving a value from one data type to another (e.g. byte → int, or int → byte). Java has two categories:

Implicit	Explicit
Java does it automatically, no code needed	You must tell Java to do it, via casting

Implicit Conversion — Widening

Rule: the **destination type must be wider** (i.e. hold a larger range) than the **source type**. If it is, Java converts automatically — no casting needed.



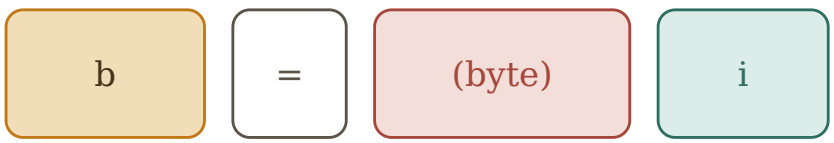
Example: `int i = b;` where b is a byte — anything a byte can hold, an int can hold too, so Java converts it silently.

This works because a smaller range (e.g. byte's −128 to +127) always fits inside a wider one (int's much larger range). The same logic applies short → int, int → long, char → int (a char, at 16 bits, fits comfortably inside a 32-bit int, using its Unicode integer value).

Widening conversion: byte → short → int → long → float → double (each step, the destination can safely hold everything the source could).

Explicit Conversion — Narrowing (Casting)

When the destination type is **smaller** than the source, Java refuses to convert automatically — because data could be lost (truncated). You get a compiler error: "*cannot convert from int to byte*". To force it anyway, you must explicitly **cast**:



Casting syntax: `byte b = (byte) i;` — you're telling the compiler this narrowing is intentional.

What Happens During Narrowing: Truncation

Example: `int i = 300; byte b = (byte) i;`. Since byte only has 8 bits, Java takes the full 32-bit binary of 300 and keeps only the **first (rightmost) 8 bits** — discarding the rest. The result: `b` becomes **44**.

Shortcut without binary: The result of narrowing a larger integer into a smaller type equals `value % (range of destination type)`. For byte (256 possible values): `300 % 256 = 44` — matches exactly.

Narrowing a char

Since char (16 bits) is smaller than int (32 bits), converting **char → int is implicit** (widening) — e.g. `int i = c;` gives you the Unicode integer value directly (97 for `'a'`).

Truncating Conversion: Decimal → Integer

Converting `float` or `double` to an integer type is a special case of narrowing called **truncating conversion** — even though float and int are both 32 bits, Java still requires an explicit cast, because int cannot hold a decimal point.

Example: `float f = 16.25f; int i = (int) f;` → the decimal part is simply dropped (not rounded) → `i = 16`.

Boolean Cannot Be Converted

Special rule: `boolean` cannot be converted to or from any other data type — implicitly or even with an explicit cast. `true` / `false` are not numbers, so there's no meaningful mapping. Attempting it (e.g. `int i = (int) myBoolean;`) is a compile-time error: "*cannot cast from boolean to int.*"

Automatic Type Promotion

A separate but related concept: whenever `byte`, `short`, or `char` values are used inside an **expression** (like a calculation), Java automatically promotes them to `int` first — even if you never wrote "int" anywhere.

Why this trips people up: `byte b = 50; b = b * 2;` gives a compile error — "cannot convert from int to byte" — even though no int appears in the code! This is because `b * 2` promotes `b` to int for the multiplication, so the result (100) is an int, and assigning an int back into a byte needs an explicit cast: `b = (byte)(b * 2);`

Type Promotion Rules

- Any `byte`, `short`, or `char` is promoted to `int` in an expression.
- If one operand is `long`, the whole expression becomes `long`.
- If one operand is `float`, the whole expression becomes `float`.
- If one operand is `double`, the whole expression becomes `double`.

Simple summary: whichever type in the expression has the largest range, the entire expression's result gets promoted to that type.

Active Recall — Part 1

Q1. What's the difference between implicit and explicit type conversion?

A. Implicit (widening) happens automatically when the destination type is wider than the source. Explicit (narrowing) requires manual casting because data could be lost.

Q2. Why does int i = 300; byte b = (byte) i; result in b = 44?

A. Only the first 8 bits of 300's 32-bit binary are kept (truncated); equivalently, 300 % 256 = 44.

Q3. Why does float f = 16.25f; int i = (int) f; give i = 16, not 17?

A. Truncating conversion simply drops the decimal part — it does not round.

Q4. Can a boolean be cast to an int?

A. No — boolean cannot be converted to/from any other type, implicitly or explicitly; there's a compile error either way.

Q5. Why does byte b = 50; b = b * 2; fail to compile?

A. Because b is automatically promoted to int for the multiplication — the result is an int, and assigning an int into a byte needs an explicit cast.

Arithmetic Operators

The familiar five: `+` `-` `*` `/` `%` (modulus — gives the remainder of division). These behave exactly as in ordinary math.

Compound Assignment Operators

Java lets you combine an operator with assignment as a shorthand: `h += 2` is equivalent to `h = h + 2`. Same pattern applies to `-=`, `*=`, `/=`, `%=`.

> **Note on `=`:** In programming, `=` never means "equals" as a comparison — it always means *assign the right-hand value to the left-hand variable*. Comparison uses a different operator (`==`), covered below.

Increment & Decrement: ++ and --

`i++` is shorthand for `i = i + 1` (and `i--` for `i = i - 1`). These are the most frequently used operators in loops, since incrementing/decrementing by exactly 1 is extremely common — unlike multiplication or division, which rarely apply by a fixed step of 1.

Prefix vs. Postfix

Both `j++` (postfix) and `++j` (prefix) increment `j` by 1 — but they differ in **when** the increment happens relative to assignment:

- **Postfix (`k = j++`):** assign j's *current* value to k first, *then* increment j.
- **Prefix (`l = ++j`):** increment j *first*, *then* assign the new value to l.

> **Example:** if j = 9 → `k = j++` gives k = 9, then j becomes 10. Next, `l = ++j` increments j to 11 first, then gives l = 11 (not the old value).

Relational Operators

`==` (equal to), `!=` (not equal to), `<`, `>`, `<=`, `>=`. Unlike `=`, these **compare** two values and always produce a `boolean` result (`true` or `false`).

> **Critical distinction:** `a == b` asks "is a equal to b?" (returns true/false). `a = b` assigns b's value into a (no comparison at all). Confusing these is one of the most common beginner bugs.

Any expression using a relational operator is called a **boolean expression** — its result can be stored directly in a `boolean` variable. These become essential once we cover conditionals (`if` statements) and loops.

Bitwise Operators

These operate at the **bit level** (0s and 1s), not on whole decimal values. The main ones:

Operator	Name	Symbol
AND	Bitwise AND	`&`
OR	Bitwise OR	`|`
XOR	Bitwise Exclusive OR	`^`
NOT	Bitwise NOT (unary)	`~`

Truth Table (per-bit rules)

A	B	A & B	A | B	A ^ B	~A
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

- **AND:** result is 1 only if *both* bits are 1.
- **OR:** result is 1 if *either* bit is 1.
- **XOR:** result is 1 only when the number of 1s is *odd* (i.e. the two bits differ).
- **NOT:** a unary operator (needs only one operand) — simply flips every bit.

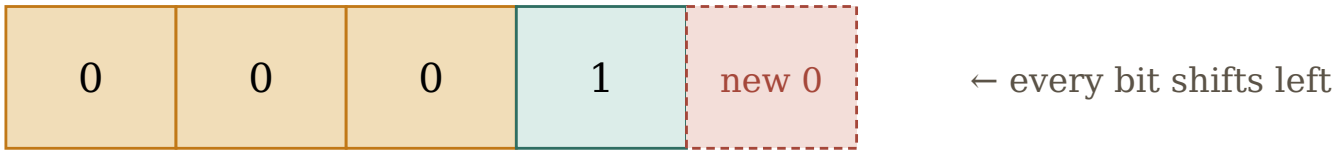
> **Example:** a = 2 (`010`), b = 3 (`011`) → a & b = 2, a | b = 3, a ^ b = 1, ~a = −3 (via two's complement, since flipping all bits of a positive number produces a negative one).

Shift Operators

Three shift operators manipulate bit positions directly: `<<` (left shift), `>>` (right shift — sign-preserving), and `>>>` (right shift with zero fill).

Left Shift (<<)

Every bit moves one position to the left; a new `0` fills the vacated rightmost slot; the leftmost bit falls off the end. **Effect: left-shifting by n is equivalent to multiplying by 2^n** — e.g. `8 << 1 = 16`, `16 << 1 = 32`.



Left shift: all bits move left one place; a fresh 0 enters at the right; the leftmost bit is discarded.

Right Shift (>>) — Sign-Preserving

All bits move right; the vacated leftmost slot is filled with a **copy of the original sign bit** (0 stays 0, 1 stays 1) — preserving whether the number was positive or negative. **Effect: equivalent to dividing by 2^n** (integer division) — e.g. `8 >> 1 = 4`.

Right Shift with Zero Fill (>>>)

Identical to `>>` except it **always** inserts a plain `0` at the leftmost slot, regardless of the original sign — meaning a negative number can become positive. Useful when negative values are meaningless in context (e.g. representing pixel color values).

Shift Operators Always Act on int (or long)

> **Critical detail:** Bitwise and shift operators never operate directly on `byte` or `short` — those are first promoted to `int` (32 bits), the shift happens on the full 32-bit value, and only afterward — if you cast the result back to byte — does truncation (keeping just the last 8 bits) occur. This is why left-shifting a byte can appear to "lose" bits that are actually still alive in the promoted int.

The Shift Limit: Mod 32 (or Mod 64 for long)

Since an int has only 32 bits, shifting by 32 or more doesn't make sense — Java automatically applies **modulus**

32 to the shift amount. So `i << 32` is equivalent to `i << 0` (no shift at all), and `i << 33` is equivalent to `i << 1`. The maximum meaningful shift is 31 places — shifting the least significant bit all the way up to the most significant bit (which flips the sign).

> **Extremes:** `1 << 31` gives `Integer.MIN_VALUE` (the smallest int a 32-bit signed integer can hold) — sometimes referred to as representing "negative infinity" territory in this context.

Logical Operators

Unlike bitwise operators (which act on bits), logical operators act on whole **boolean expressions**: `&&` (logical AND) and `||` (logical OR), following the same truth-table logic as their bitwise counterparts, but applied to `true` / `false` results instead of 0/1 bits.

Short-Circuit Evaluation

Java's `&&` and `||` are "smart" — they skip evaluating the second expression when the result is already determined:

- **AND (`&&`):** if the first expression is `false`, the whole thing must be `false` — so the second expression is never evaluated.
- **OR (`||`):** if the first expression is `true`, the whole thing must be `true` — so the second expression is never evaluated.

> **Why it matters:** Short-circuiting is heavily used in real code — e.g. to safely check a condition before evaluating something that would fail otherwise (like checking an object isn't null before using it).

> **Interview detail:** The single-character bitwise `&` and `|` can technically also be used between two boolean expressions (not just bits) — giving the same true/false result as `&&` / `||`. But they do **NOT** short-circuit — both sides always get evaluated. This distinction is a well-known interview question; in real code, `&&` and `||` are used almost exclusively.

Assignment Operator

`=` assigns the right-hand value into the left-hand variable. It can be chained: `int a = b = c = 10;` evaluates right-to-left — 10 goes into c, then c's value into b, then b's value into a — giving all three variables the value 10.

Ternary Operator (Preview)

Java has exactly one operator that takes **three operands** — the ternary operator. Unlike unary operators (one operand, e.g. `~a`) or binary operators (two operands, e.g. `a + b`), the ternary operator needs three. It's most useful alongside conditionals (`if` / `else`), so a full explanation is saved for that topic.

Operator Precedence

Just like BODMAS in math, Java evaluates operators in a fixed precedence order when multiple appear in one expression — postfix operators highest, then prefix, then arithmetic (`*` `/` `%` before `+` `-`), then shifts, then relational, then bitwise, then logical, with the assignment operator (`=`) evaluated **last** (and right-to-left).

> **Practical advice:** Rather than memorizing the full precedence table, use **parentheses** to make evaluation order explicit — parentheses have the highest precedence of all and let you control exactly how a complex expression is evaluated. Real-world code almost always favors clarity via brackets over relying on memorized precedence rules.

Active Recall — Part 2

Q1. What's the difference between postfix (j++) and prefix (++j) increment?

A. Postfix assigns the current value first, then increments. Prefix increments first, then assigns the new value.

Q2. What's the difference between = and ==?

A. = is the assignment operator (assigns a value to a variable). == is the relational equality operator (compares two values and returns true/false).

Q3. What does the bitwise XOR (^) operator return?

A. 1 (true) only when the number of 1-bits among the operands is odd; otherwise 0.

Q4. What does left-shifting a number by n places do mathematically?

A. It's equivalent to multiplying the number by 2^n.

Q5. Why do bitwise/shift operators on a byte still behave "correctly" even though bytes are only 8 bits?

A. Because the byte is first promoted to a 32-bit int, the operation happens on the full int, and only the final cast back to byte truncates to 8 bits.

Q6. What's the difference between >> and >>>?

A. >> (signed right shift) fills the vacated leftmost bit with a copy of the sign bit, preserving negative/positive. >>> always fills with 0, which can turn a negative number positive.

Q7. What does short-circuit evaluation mean for && and ||, and which operators lack it?

A. && skips checking the second condition if the first is false; || skips it if the first is true. The single-character & and | do NOT short-circuit — both sides are always evaluated.

Q8. What's the safest way to control evaluation order in a complex expression instead of memorizing precedence rules?

A. Use parentheses — they have the highest precedence and make the intended order explicit and readable.
